



**INSTITUTO FEDERAL  
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**  
Minas Gerais  
Campus Sabará

**Disciplina: Redes de Computadores II**

**Curso: BSI**

**Professor: Renato Miranda Filho**

### **TRABALHO PRÁTICO III**

O objetivo deste trabalho é desenvolver uma aplicação cliente-servidor utilizando Python e sockets para simular um sistema de *matchmaking* de uma plataforma de jogos online. Não é necessário implementar interface gráfica. O trabalho poderá ser realizado individualmente ou em dupla.

#### **Contexto**

Plataformas de jogos online utilizam servidores para organizar jogadores interessados em participar de partidas. Quando um jogador deseja jogar, ele envia uma solicitação ao servidor, que o coloca em uma fila de espera até que existam jogadores suficientes para formar uma partida. Neste trabalho, vocês deverão implementar uma versão simplificada desse processo.

#### **Descrição do Sistema**

O sistema será composto por dois programas:

Cliente (Jogador)

Servidor (Sistema de Matchmaking)

#### **Cliente (Jogador)**

O cliente representa um jogador que deseja participar de uma partida online.

Ao ser executado, o programa deverá solicitar ao usuário:

Nome do jogador;

Nível do jogador (1 a 100);

Jogo desejado.

Exemplos de jogos:

Valorant

League of Legends

Counter-Strike

FIFA

Minecraft

Após coletar as informações, o cliente deverá conectar-se ao servidor e enviar os dados.

Em seguida, deverá aguardar e exibir a resposta enviada pelo servidor.

#### **Servidor (Sistema de Matchmaking)**

O servidor deverá permanecer em execução aguardando conexões de clientes. Não é necessário atender vários clientes simultaneamente. O servidor poderá atender um cliente por vez, processando uma solicitação, enviando a resposta e voltando a aguardar uma nova conexão.

#### **1. Gerenciamento das Filas**

O servidor deverá manter uma fila de espera para cada jogo disponível.

Exemplo:

Fila Valorant:

João  
Maria  
Carlos

Fila FIFA:

Pedro  
Ana

## 2. Formação de Partidas

Cada jogo deverá possuir uma quantidade mínima de jogadores para que uma partida seja criada.

Utilize os seguintes valores:

| Jogo              | Jogadores necessários |
|-------------------|-----------------------|
| Valorant          | 4                     |
| League of Legends | 5                     |
| Counter-Strike    | 4                     |
| FIFA              | 2                     |
| Minecraft         | 3                     |

Sempre que um novo jogador entrar em uma fila, o servidor deverá verificar se existe quantidade suficiente de jogadores para criar uma partida.

### Caso exista:

Os jogadores necessários deverão ser removidos da fila;

Uma nova partida deverá ser criada;

A partida deverá receber um identificador

Exemplo:

Partida criada!  
Jogo: Valorant  
ID da partida: 103  
Participantes:  
João  
Maria  
Carlos  
Pedro

## 3. Resposta ao Cliente

Após receber a solicitação, o servidor deverá responder uma das seguintes mensagens.

*Situação 1 – Jogador adicionado à fila*

Exemplo:

Você foi adicionado à fila de espera.

Jogo: Valorant

## *Situação 2 – Partida criada*

Exemplo:

Uma nova partida foi criada.

Jogo: FIFA

ID da partida: 208

2 jogadores foram selecionados

### **Requisitos Obrigatórios**

O trabalho deverá obrigatoriamente utilizar:

Linguagem Python;

Biblioteca socket;

Arquitetura cliente-servidor.

Não será permitido utilizar frameworks que abstraíam completamente o uso de sockets.

Entregar

1. Código-Fonte (O código deverá estar organizado e comentado)
  - a. Do cliente
  - b. Do servidor

As principais partes da solução deverão possuir comentários explicativos.

2. Relatório Técnico

O relatório deverá conter:

- a) Descrição da solução
- b) Casos de Teste

Apresentar capturas de tela dos casos de teste demonstrando:

Conexão entre cliente e servidor;

Inclusão de jogadores em filas;

Consulta da posição na fila;

Criação automática de uma partida quando houver jogadores suficientes;

Remoção correta dos jogadores da fila;

**Cada captura deverá conter uma breve explicação.**